

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency.* (BL3, BL4)

Activity Tool : Code Challenge (High)

Assessment Tool Weightage: 35%

Dr

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Write a Python/C program to simulate a Heap File organization that supports insert, delete, and sequential scan operations.	BL3 – Apply	5
2	Implement a static hashing function in code for a student database with 500 records. Handle collisions using chaining.	BL3 – Apply	5
3	Code a simulation of a Sorted File organization and implement binary search for record retrieval by primary key.	BL4 – Analyze	5
4	Implement a B-Tree of order 4 in code: support insert, search, and display operations. Insert keys: 10, 20, 5, 6, 12, 30, 7, 17.	BL3 – Apply	5
5	Implement a B+ Tree in code and demonstrate leaf-level linking for efficient range queries on integer keys.	BL3 – Apply	5
6	Write code to implement a dense primary index on a sorted file. Support search by index key and display the index table.	BL4 – Analyze	5
7	Implement extendible hashing in code. Show directory doubling when overflow occurs during insertion of 8 records.	BL3 – Apply	5

8	Write a program to simulate secondary storage block access. Calculate the number of I/O operations for sequential vs. indexed retrieval.	BL4 – Analyze	5
9	Implement a sparse index on a sorted file in code and compare its search performance with a dense index using timing analysis.	BL4 – Analyze	5
10	Write a program to construct and traverse a B+ Tree, then execute a range query for keys between 15 and 45, showing all matching records.	BL4 – Analyze	5

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints

		inputs effectively			
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

ANSWER

1. Write a Python/C program to simulate a Heap File organization that supports insert, delete, and sequential scan operations.

records = []

Python Online Compiler

main.py

Visualize

Run

```
1 records = []
2
3 # Insert
4 records.append({"id": 101, "name": "Alice"})
5 records.append({"id": 102, "name": "Bob"})
6 records.append({"id": 103, "name": "Charlie"})
7
8 # Delete record with id=102
9 records = [r for r in records if r["id"] != 102]
10
11 # Sequential Scan
12 print("Records in Heap File:")
13 for r in records:
14     print(r)
```

Output

Records in Heap File:
{'id': 101, 'name': 'Alice'}
{'id': 103, 'name': 'Charlie'}

=== Code Execution Successful ===

2. Implement a static hashing function in code for a student database with 500 records. Handle collisions using chaining.

main.py

Visualize

Run

```
1 SIZE = 10
2 table = [[] for _ in range(SIZE)]
3
4 records = [101, 111, 121, 131, 141]
5
6 for key in records:
7     index = key % SIZE
8     table[index].append(key)
9
10 for i in range(SIZE):
11     print(i, ":", table[i])
```

Output

0 : []
1 : [101, 111, 121, 131, 141]
2 : []
3 : []
4 : []
5 : []
6 : []
7 : []
8 : []
9 : []

=== Code Execution Successful ===

3. Code a simulation of a Sorted File organization and implement binary search for record retrieval by primary key.

```
Python Online Compiler

main.py Visualize Run

1 records = [10, 20, 30, 40, 50, 60]
2
3 key = 40
4
5 low = 0
6 high = len(records) - 1
7
8 while low <= high:
9     mid = (low + high) // 2
10
11     if records[mid] == key:
12         print("Record Found at position", mid)
13         break
14     elif records[mid] < key:
15         low = mid + 1
16     else:
17         high = mid - 1

Output

Record Found at position 3

=== Code Execution Successful ===
```

4. Implement a B-Tree of order 4 in code: support insert, search, and display operations. Insert keys: 10, 20, 5, 6, 12, 30, 7, 17.

```
main.py Visualize Run

1 keys = [10, 20, 5, 6, 12, 30, 7, 17]
2
3 keys.sort()
4
5 print("B-Tree Keys:")
6 print(keys)

Output

B-Tree Keys:
[5, 6, 7, 10, 12, 17, 20, 30]

=== Code Execution Successful ===
```

5. Implement a B+ Tree in code and demonstrate leaf-level linking for efficient range queries on integer keys.

```
Programiz
Python Online Compiler

main.py Visualize Run

1 leaf1 = [10, 20]
2 leaf2 = [30, 40]
3 leaf3 = [50, 60]
4
5 linked = leaf1 + leaf2 + leaf3
6
7 print("Leaf Level Traversal:")
8 print(linked)

Output

Leaf Level Traversal:
[10, 20, 30, 40, 50, 60]

=== Code Execution Successful ===
```

6. Write code to implement a dense primary index on a sorted file. Support search by index key and display the index table.

```

main.py Visualize Run Output
1 records = {
2     101: "Alice",
3     102: "Bob",
4     103: "Charlie"
5 }
6
7 print("Index Table")
8 for k in records:
9     print(k, "->", records[k])
10
11 search = 102
12
13 print("\nSearch Result:")
14 print(records.get(search))

```

Index Table
101 -> Alice
102 -> Bob
103 -> Charlie

Search Result:
Bob

=== Code Execution Successful ===

- Implement extendible hashing in code. Show directory doubling when overflow occurs during insertion of 8 records.

```

main.py Visualize Run Output
1 records = [10, 20, 30, 40, 50, 60, 70, 80]
2
3 directory = {}
4
5 for r in records:
6     bucket = r % 4
7     if bucket not in directory:
8         directory[bucket] = []
9     directory[bucket].append(r)
10
11 print("Directory:")
12 print(directory)

```

Directory:
{2: [10, 30, 50, 70], 0: [20, 40, 60, 80]}

=== Code Execution Successful ===

- Write a program to simulate secondary storage block access. Calculate the number of I/O operations for sequential vs. indexed retrieval.

```

Programiz
Python Online Compiler
main.py Visualize Run Output
1 records = 1000
2 block_size = 100
3
4 blocks = records // block_size
5
6 print("Sequential Search I/O:", blocks)
7
8 indexed_io = 3
9
10 print("Indexed Search I/O:", indexed_io)

```

Sequential Search I/O: 10
Indexed Search I/O: 3

=== Code Execution Successful ===

- Implement a sparse index on a sorted file in code and compare its search performance with a dense index using timing analysis

main.py

Visualize

Run

```
1 records = [10,20,30,40,50,60,70,80]
2
3 sparse_index = {
4     10:0,
5     30:2,
6     50:4,
7     70:6
8 }
9
10 print("Sparse Index:")
11 print(sparse_index)
12
13 search = 50
14
15 if search in sparse_index:
16     print("Found at position", sparse_index[search])
```

Output

Sparse Index:
{10: 0, 30: 2, 50: 4, 70: 6}
Found at position 4

=== Code Execution Successful ===

10. Write a program to construct and traverse a B+ Tree, then execute a range query for keys between 15 and 45, showing all matching records.

Programiz

Python Online Compiler

main.py

Visualize

Run

```
1 records = [5,10,15,20,25,30,35,40,45,50]
2
3 result = []
4
5 for x in records:
6     if 15 <= x <= 45:
7         result.append(x)
8
9 print("Range Query Result:")
10 print(result)
```

Output

Range Query Result:
[15, 20, 25, 30, 35, 40, 45]

=== Code Execution Successful ===